PreLAB: Timer Interrupt & PWM

Date: 2026-11-2

Author/Partner: Juyoung Lee (22200576)

Introduction

In this prelab, you will learn how to set up MCU Timers for Timer Interrupt and PWM output.

Part 1: Timer Interrupt

A. Register List

List of TIMx registers for Timer Interrupt

Type	Register Name	Description
TIMx	TIMx_ CR1	TIMx control register 1
	TIMx_ PSC	TIMx prescaler register
	TIMx_ARR	TIMx auto-reload register
	TIMx_DIER	TIMx DMA Interrupt Enable register

B. Register Setting

1. **System Clock setting**
   - RCC setting (PLL)
2. **Timer Counter setting**
   - Enable Timer Peripheral Clock (RCC->APB1ENR)
   - Set Timer Clock Pre-scaler value (TIMx->PSC : PSC[15:0])
   - Set Auto-reload value (TIMx->ARR : ARR)
   - Set Counting Direction (TIMx->CR1 : DIR)
   - Enable counter (TIMx->CR1 : CEN)
3. **Timer Interrupt setting**
   - Enable Timer DMA/Interrupt. (TIMx->DIER : UIE)
4. **NVIC setting**
   - Set Interrupt Priority NVIC_SetPriority(TIMx_IRQn,2)
   - Enable TIMx Interrupt: NVIC_EnableIRQ(TIMx_IRQn)

C. Exercise

Exercise: TIMER Library

- Download the header library files and save under `include\`.
    - `ecTIM2_student.h, ecTIM2_student.c`: Click here to download
- Rename the files as `ecTIM2.h, ecTIM2.c`

> Write your name and modified date in the comment box

- Fill in the empty spaces in the downloaded library: `ecTIM2.c`
- Run the program and check your result.
- **Submit** the exercise code in the report

Exercise Code

{% code expandable="true" %}

```
// Exercises are in these functions.
void TIM_init(TIM_TypeDef* TIMx, uint32_t msec);
void TIM_init_us(TIM_TypeDef* TIMx, uint32_t usec);
void TIM_period_us(TIM_TypeDef* TIMx, uint32_t usec);
```

{% endcode %}

{% tabs %}
{% tab title="Exercise" %}
{% code expandable="true" %}

```
////////////////////////////////////////////////////////////////
/* Timer Counter Initialization                               */
////////////////////////////////////////////////////////////////


// Timer Counter Initialization
// Default : f_cnt =  10kHz, msec= 1 to 6,553 msec
void TIM_init(TIM_TypeDef* TIMx, uint32_t msec){
// 1. Enable Timer CLOCK
    TIM_clock_enable(TIMx);

// 2. Set CNT period
    TIM_period(TIMx, msec);

// 3. COUNTER Direction
    // Up-Counter
    TIMx->CR1 &= ~TIM_CR1_DIR;

    // // Down-Counter
    // TIMx->CR1 |= TIM_CR1_DIR;

// 4. Enable Timer Counter
```

```
    TIMx->CR1 |= TIM_CR1_CEN;
}


// Timer Counter Initialization
// Default : f_cnt =  1 MHz, usec= 1 to 65,530 usec
void TIM_init_us(TIM_TypeDef* TIMx, uint32_t usec){
// 1. Enable Timer CLOCK
    TIM_clock_enable(TIMx);

// 2. Set CNT period
    TIM_period_us(TIMx, usec);

// 3. CNT Direction
    // Up-Counter
    TIMx->CR1 &= ~TIM_CR1_DIR;

    // Down-Counter
    TIMx->CR1 |= TIM_CR1_DIR;

// 4. Enable Timer Counter
    TIMx->CR1 |= TIM_CR1_CEN;
}


// Limits    : 16-bit ARR -> f_cnt =  10kHz, msec= 1 to 6,553 msec (ARR < 0xFFFF)
void TIM_period(TIM_TypeDef* TIMx, uint32_t msec){

    uint32_t Sys_CLK = EC_SYSCLK;                   // 84MHz (PLL) or 16MHz (HSI)

    // Counter Frequency (f_cnt)
    uint32_t f_cnt   =  10000UL;                    // 16-bit CNT: f_cnt=10kHz (default)

    // Prescaler Value
    uint32_t PSCval = Sys_CLK / f_cnt;              // 84MHz -> 10 kHz (PSC=8400)
    TIMx->PSC = PSCval - 1;

    // Reload Value (ARR) for msec unit
    uint32_t ARRval = (f_cnt / 1000UL) * msec;      // Reload values needed for msec
    TIMx->ARR = ARRval - 1;
}


// Limits    : 16-bit ARR -> f_cnt =  1 MHz, usec= 1 to 65,530 usec (ARR < 0xFFFF)
void TIM_period_us(TIM_TypeDef* TIMx, uint32_t usec){

    uint32_t Sys_CLK = EC_SYSCLK;                   // 84MHz (PLL) or 16MHz (HSI)

    // Counter Frequency (f_cnt)
    uint32_t f_cnt   =  1000000UL;                  // 16-bit CNT: f_cnt=1MHz (default)

    // Prescaler Value
    uint32_t PSCval = Sys_CLK / f_cnt;              // 84MHz -> 1 MHz (PSC=84)
```

```
    TIMx->PSC = PSCval - 1;
    // [YOUR CODE GOES HERE !!!!]

    // Reload Value (ARR) for msec unit
    uint32_t ARRval = (f_cnt / 1000000UL) * usec;
    // [YOUR CODE GOES HERE !!!!]
    TIMx->ARR = ARRval - 1;
}
```

{% endcode %}
{% endtab %}

{% tab title="ecTIM2.h" %}

```
////////////////////////////////////////////
/* Timer Update Interrupt Configuration    */
////////////////////////////////////////////

// Timer Update Interrupt Initialization
void TIM_UI_init(TIM_TypeDef* TIMx, uint32_t msec);     // 16-bit  msec < 65,530 msec
void TIM_UI_init_us(TIM_TypeDef* TIMx, uint32_t usec);  // 16-bit  usec < 65,530 usec

// Timer Update Interrupt On/OFf
void TIM_UI_enable(TIM_TypeDef* TIMx);
void TIM_UI_disable(TIM_TypeDef* TIMx);

// Timer Update Interrupt Flag
uint32_t is_UIF(TIM_TypeDef *TIMx);
void clear_UIF(TIM_TypeDef *TIMx);


////////////////////////////////////////////////////////////
/* Timer Counter Initialization                           */
////////////////////////////////////////////////////////////

// Timer Counter Initialization
void TIM_init(TIM_TypeDef *TIMx, uint32_t msec);
void TIM_init_us(TIM_TypeDef *TIMx, uint32_t usec);

// Timer Counter Period Config
void TIM_period(TIM_TypeDef* TIMx, uint32_t msec);
void TIM_period_ms(TIM_TypeDef* TIMx, uint32_t msec);   // same as TIM_period()
void TIM_period_us(TIM_TypeDef* TIMx, uint32_t usec);

// Timer Counter Auxiliary Functions
uint32_t get_TIM_IRQn(TIM_TypeDef* TIMx);
void TIM_clock_enable(TIM_TypeDef* TIMx);
```

{% endtab %}
{% endtabs %}

Exercise: TIMER application

This is an example code for Timer Interrupt that turns LED on/off at 1 second period.

Configuration

- TIM2: Up-counting, Timer2_CLK = 100 kHz, COUNT_CLK = 1 kHz
- Use timer interrupt “void TIM2_IRQHandler(void)”

Procedure

- Create a new project under the directory `EC\tutorial\`
    - Environment : `env:TU_TIM_Interrupt`
    - Source file: `TU_TIM_Interrupt_student.c`
- Modify the `platformio.ini` , to add new environment.
- Run the program and check your result.
- No need to submit the result in the prelab report
- But you need to show the demonstration in class.

{% tabs %}
{% tab title="TU_TIM_Interrupt_student.c" %}
{% code expandable="true" %}

```
/*----------------------------------------------------------------\
@ Embedded Controller by Young-Keun Kim - Handong Global University
Author           : [ YOUR NAME GOES HERE !!!!!]
Created          : 05-03-2021
Modified         : 09-03-2026 [WRITE THE DATE!!!!]
Language/ver     : C++ in VS Code

Description      : [Write description here!!]
/----------------------------------------------------------------*/


// #include "ecSTM32F411.h"
#include "stm32f411xe.h"
#include "ecRCC2.h"
#include "ecGPIO2.h"
#include "ecSysTick2.h"
#include "ecTIM2.h"


#define LD2_PIN PA_5
uint32_t _count = 0;
// led_toggle() function should have been defined in ecGPIO2.c


// Initialization
void setup(void){
    RCC_PLL_init();
    SysTick_init();
```

```
    GPIO_init(LD2_PIN, OUTPUT);

    // TIM2 Update-Event Interrupt every 1 msec
    TIM_UI_init(TIM2, 1);           // UEV every 1 msec
    //TIM_UI_init_us(TIM2, 1000);   // UEV every 1000usec=1msec
}

int main(void) {
    // Initialization --------------------------------------------------
    setup();

    // Infinite Loop ---------------------------------------------------
    while(1){}
}


// TIM2 Update Interrupt Handler
void TIM2_IRQHandler(void){
    if(is_UIF(TIM2)){           // Check UIF(update interrupt flag)
        _count++;
        if (_count > 1000) {
            led_toggle(LD2_PIN);    // Toggle every 1000 msec
            _count = 0;
        }
        clear_UIF(TIM2);        // Clear TIM_UI flag
    }
}
```

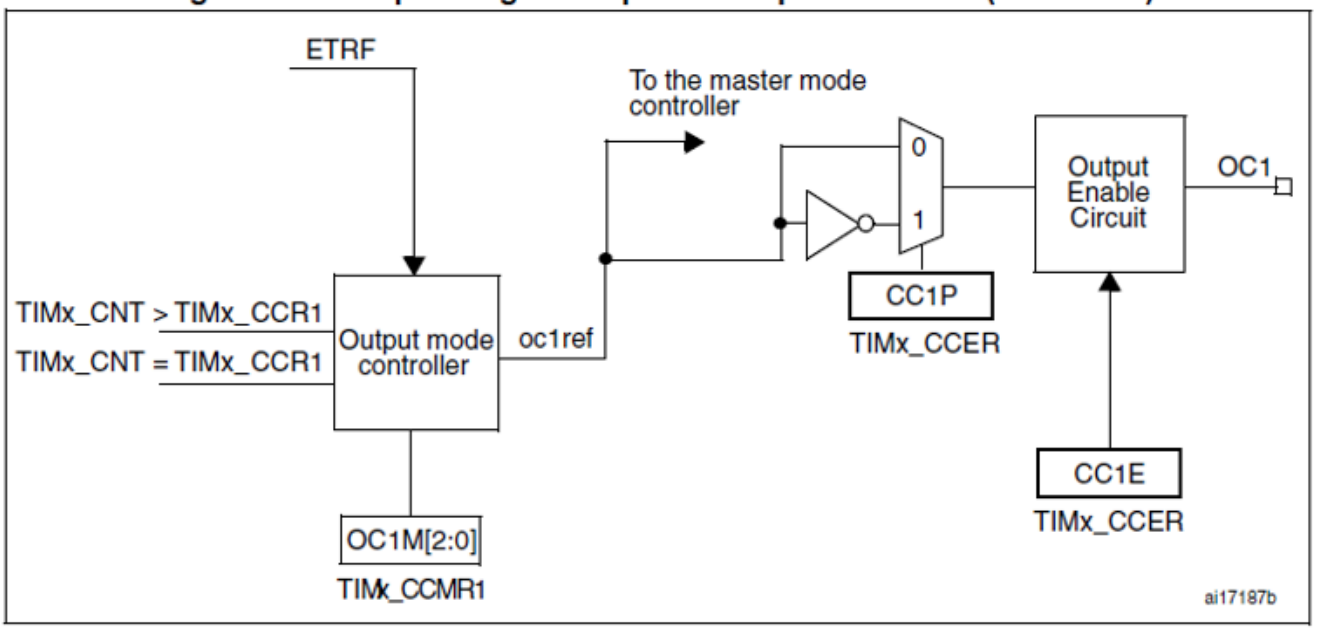
{% endcode %}
{% endtab %}
{% endtabs %}

Part 2: PWM

A. Register List

List of TIMx registers for PWM

Figure 114. Output stage of capture/compare channel (channel 1)



Type	Register Name	Description
TIMx	TIMx _CCMRy	TIMx capture/compare mode register for yth channel
	TIMx _CCRy	TIMx capture/compare register for yth channel
	TIMx _CCER	TIMx capture/compare output enable register

B. Register Setting

1. System Clock setting

- Same as above

2. Timer Counter setting

- Same as above

3. **Timer Interrupt setting**

- If necessary

4. GPIO (AF) Out

- mode=AF(TIMx) for Pin_y in GPIOx

5. PWM Out setting

- Select PWM Output mode (TIMx->CCMR : OCyM)
- Select CompareCapture value (TIMx->CCRy : CCR)
- Select Output Polarity (TIMx->CCER : CCyP)
- Enable CompareCaptureOutput (TIMx->CCER : CCyE)

C. Exercise

Exercise: PWM Library

- Download the header library files and save under `include\`.
    - `ecPWM2_student.h, ecPWM2_student.c`: Click here to download
- Rename the files as `ecPWM2.h, ecPWM2.c`

> Write your name and modified date in the comment box

- Fill in the empty spaces in the downloaded library: `ecPWM2.c`
- Submit this tutorial report that shows the **Exercise Code**.

Exercise Code

{% code expandable="true" %}

```
// Internal Helper function: for PWM Initialization
void PWM_init_config(PinName_t pinName, uint32_t usec);
// Duty ratio 0~1.0
void PWM_duty(PinName_t pinName, float duty);
```

{% endcode %}

{% tabs %}
{% tab title="Exercise" %}
{% code expandable="true" %}

```
////////////////////////////////////////////////
/*              EXERCISE                      */
////////////////////////////////////////////////


// Internal Helper function: for PWM Initialization
void PWM_init_config(PinName_t pinName, uint32_t usec){
// 0. Match TIMx from given Port_y and Pin_x
    GPIO_TypeDef *port;
    unsigned int pin;
    ecPinmap(pinName, &port, &pin);
    TIM_TypeDef *TIMx;
    int chN;
    PWM_pinmap(pinName, &TIMx, &chN);

// 1. Initialize GPIO port and pin as AF
    GPIO_init(pinName, AF);

// 2. Configure GPIO AFR by Pin x num.
    //  AFR[0] holds pin 0~7, AFR[1] holds pin 8~15 : 4 bits per pin
        uint32_t afrIdx   = pin >> 3;               // same as (pin/8)
    uint32_t afrShift = 4 * (pin % 8);
    uint32_t afNum    = 0;
```

```
    //  AF1 for TIM1,TIM2     AF2 for TIM3,TIM4
    if((TIMx == TIM1)||(TIMx == TIM2))                      afNum = 1;  // AF1
    else if(TIMx== TIM3 || TIMx == TIM4)                    afNum = 2;  // AF2
    else if(TIMx == TIM9 || TIMx == TIM10 || TIMx == TIM11) afNum = 3;  // AF3
    // You can add more  for TIM5 to TIM11, if necessary [YOUR CODE GOES HERE!!]


    // Set AFR register for the pin
    port->AFR[afrIdx] &= ~(0xFUL << afrShift);  // 4 bits per pin
    port->AFR[afrIdx] |= (afNum << afrShift)  ;



// 3. Initialize Timer : select CLK_CNT according to PWM period
    //   TIM2/TIM5          (32-bit ARR) : 1 < usec < 2^32 usec     -> TIM_init_us()
    //   TIM1/TIM3/TIM4     (16-bit ARR) : 1 < usec <=65,536 usec   -> TIM_init_us()
    //                                     65 < msec <65,536 msec  -> TIM_init()
    if((TIMx == TIM2) || (TIMx == TIM5) || (usec <= 65536)){
        TIM_init_us(TIMx, usec);
    }
    else {
        uint32_t msec = usec / 1000;
        if(msec > 6553) msec = 6553;            // 16-bit ARR has limit of 6.5sec
                                                // 65 < msec <65,536 msec   -> TIM_init()
        TIM_init(TIMx, msec);
    }

// 4. Configure Timer Output mode as PWM
    // Default: PWM Mode 1, active high, duty ratio = 50%
    PWM_mode_config(TIMx, chN, EC_PWM_MODE1);

// 5. Enable Timer Counter
    // For TIM1 ONLY
    if(TIMx == TIM1) TIMx->BDTR |= TIM_BDTR_MOE;                    // Main output enable
(MOE): 0 = Disable, 1 = Enable
    // All Timer: First, Clear the counter and Clear UI Flag
    TIMx->EGR |= TIM_EGR_UG;
    TIMx->SR   = ~TIM_SR_UIF;
    // Enable timers
    TIMx->CR1  |= TIM_CR1_CEN;

}


// Duty Ratio:  from 0.0 to 1.0
void PWM_duty(PinName_t pinName, float duty){
// 0. Match TIMx from  Port and Pin
    GPIO_TypeDef *port;
    unsigned int pin;
    ecPinmap(pinName, &port, &pin);
    TIM_TypeDef *TIMx;
    int chN;
    PWM_pinmap(pinName, &TIMx, &chN);
```

```
// 1. Check the duty ratio range from 0.0 to 1.0
    if(duty < 0.0f) duty = 0.0f;
    else if (duty > 1.0f) duty = 1.0f;

// 2. Duty ratio -> CCR counts.  Assume PWM mode 1 :
    uint32_t CCvalue = (uint32_t)((TIMx->ARR + 1) * duty);  // (ARR+1) * duty



    if(chN == 1)      { TIMx->CCR1 = CCvalue; } //set channel
    else if(chN == 2) { TIMx->CCR2 = CCvalue; }
    else if(chN == 3) { TIMx->CCR3 = CCvalue; }
    else if(chN == 4) { TIMx->CCR4 = CCvalue; }


}
```

{% endcode %}
{% endtab %}

{% tab title="ecPWM2.h" %}

```
////////////////////////////////////////////////
/*              EXERCISE                      */
////////////////////////////////////////////////

// Internal Helper function: for PWM Initialization
void PWM_init_config(PinName_t pinName, uint32_t usec);
// Duty ratio 0~1.0
void PWM_duty(PinName_t pinName, float duty);


////////////////////////////////////////////////
/*      GIVEN:  PWM Initialization              */
////////////////////////////////////////////////

//  Low Frequency PWM initialization in msec. (below 1kHz)
//  Default:        50% duty ratio. PWM_mode1, Period in msec.
//  TIM2/TIM5       (32-bit ARR) : msec 1 ~ 4,294,967
//  TIM1/TIM3/TIM4  (16-bit ARR) : msec 1 ~ 6,553
void PWM_init(PinName_t pinName, uint32_t msec);    // 1 to 6,553 msec (16-bit)

//  High-Frequency PWM initialization in usec (1kHz to 100 kHz)
//  Default:        50% duty ratio.  PWM_mode1, Period in usec.
//  TIM2/TIM5       (32-bit ARR) :  usec 1 ~ 4,294,967
//  TIM1/TIM3/TIM4  (16-bit ARR) :  usec 1 ~ 6,553
void PWM_init_us(PinName_t pinName, uint32_t usec); // 10 usec and up


////////////////////////////////////////////////
/*      GIVEN: DUTY RATIO SETUP                */
```

```
////////////////////////////////////////////////

// Set Pulse width in msec
void PWM_pulsewidth(PinName_t pinName, uint32_t pulse_width_ms);

// Set Pulse width in usec
void PWM_pulsewidth_us(PinName_t pinName, uint32_t pulse_width_us);
```

{% endtab %}
{% endtabs %}

Exercise: PWM Application

This is an example code for PWM Out

- Give PWM output of 0% to 100% a step of 20%
- Check the brightness of LD2 (PA5) for each different Duty ratio

Configuration

- System CLK is PLL 84MHz
- Timer Counter:
  - TIM2: Up-counting, Timer2_CLK = 100 kHz, COUNT_CLK=1 kHz
- PWM Output:
  - PWM Out on PA_5 : TIM2_CH1
  - PWM Mode 1 / Period 1kHz / Duty ratio 50%.

> There are several pins for TIM2_CH1. We will use GPIO Pin5 (LD2) for tutorial
>
> MCU pins for TIMERS

Procedure

- Create a new project under the directory `EC\tutorial\`
  - Environment : `env:TU_TIM_PWM`
  - Source file: `TU_TIM_PWM_student.c`
- Modify the `platformio.ini` , to add new environment.
- Run the program and check your result.
- No need to submit the result in the prelab report
- But you need to show the demonstration in class.

{% code expandable="true" %}

```
/*----------------------------------------------------------------@ Embedded Controller by
Young-Keun Kim - Handong Global University
Author           : [ YOUR NAME GOES HERE !!!!!]
Created          : 10-13-2026
Modified         : 00-00-2026 [WRITE THE DATE!!!!]
Language/ver     : C in VS Code
```

```
Description      : Tutorial - PWM
                   PA_5 (TIM2_CH1, Nucleo LD2) : 100 Hz, duty sweep 0 -> 100%

/----------------------------------------------------------------*/

// #include "ecSTM32F411.h"
#include "ecGPIO2.h"
#include "ecSysTick2.h"
#include "ecRCC2.h"
#include "ecTIM2.h"
#include "ecPWM2.h"

// Button Pin & PWM Pin
#define PWM_LED PA_5

// Initialiization
void setup(void) {
    RCC_PLL_init();
    SysTick_init();

    // PWM of 10 msec:  TIM2_CH1 (PA_5)
    PWM_init(PWM_LED, 10);          // 10 msec period -> 100 Hz
    PWM_duty(PWM_LED, 0.2f);        // 20% of duty
    // PWM_pulsewidth(PWM_LED, 2); //  a Pulse of 2msec =(duty=1/10=20%)

    // PWM of 1 msec:  TIM2_CH1 (PA_5)
    // PWM_init_us(PWM_LED, 1000);      // 1 msec period -> 1 kHz
    // PWM_duty(PWM_LED, 0.2f);     // 20% of duty
}



int main(void) {
    // Initialization --------------------------------------------------
    setup();

    // Infinite Loop ---------------------------------------------------
    while(1){
        for (int i=0; i<5; i++) {
            PWM_duty(PWM_LED, (float)0.2*i);
            delay_ms(1000);
        }
    }
}
```

{% endcode %}

####